Excellent — here's your complete breakdown for **Week 5 – GenAI APIs (OpenAI, Gemini, Claude, Mistral)**. This week equips you with real-world **API integration skills** — a must for building smart, scalable AI apps that plug into production environments.

□ WEEK 5 – GenAI APIs (OpenAI, Gemini, Claude, Mistral)

□ 1. OpenAI API

Subtopics:

- How OpenAI API works (GPT-4, GPT-3.5, Whisper, DALL·E, etc.)
- API keys, rate limits, safety filters
- Using `openai` Python SDK
- Parameters: temperature, max_tokens, top_p, stop
- Completion vs Chat models
- Streaming output
- Pricing and tokens (cost optimization tips)

□ 2. Gemini API (Google's Generative Models)

Subtopics:

- What is Gemini 1.5 / Gemini Pro?
- Gemini vs GPT-4: comparison
- Google AI Studio vs Vertex AI
- Setting up API access via MakerSuite
- Sending text, image prompts (multimodal)
- Using Gemini in Python
- Safety filters and system prompts

□ 3. Claude API (Anthropic)

Subtopics:

- Claude 3 family (Opus, Sonnet, Haiku)

- Claude vs GPT-4 vs Gemini
- Anthropic's focus on safety, honesty, harmlessness
- Setting up API via Claude Console
- Structure of Claude messages (`system`, `user`, `assistant`)
- Use cases: summarization, compliance, reasoning
- Prompting strategies (Constitutional AI, system priming)

□ 4. Mistral API or Self-Hosted Models

Subtopics:

- What is Mistral 7B / Mixtral?
- Differences: open-weight vs proprietary (OpenAI)
- Hosting with Hugging Face or Ollama
- Quantized models (GGUF, GPTQ)
- Run Mistral locally with LangChain or LlamaIndex
- Pros and cons of self-hosting

□□ 5. API Integration in Python Projects

Subtopics:

- Using requests or SDKs (`openai`, `google.generativeai`, `anthropic`)
- Structuring prompt templates in code
- Passing dynamic input from user / form
- Retry logic, timeout, and error handling
- Logging + debugging prompt outputs
- Creating your own wrapper class for each model

□ 6. Comparing GenAI APIs

Subtopics:

- Speed, latency, response quality (Claude vs GPT vs Gemini)
- Best use cases per API (chat, vision, document Q&A, summarization)
- Price comparison by 1K tokens
- API limitations and safety controls
- Enterprise features (team API keys, audit logging, sandboxing)

□ 7. Multimodal GenAI APIs

Subtopics:

- OpenAI: ChatGPT Vision, DALL·E image generation/editing
- Gemini: text + image + video + audio (multimodal input/output)
- Using vision models to caption, reason, classify
- Combine image + text in prompts
- Multimodal applications: resume scanner, medical assistant, diagram Q&A

□ OPTIONAL BONUS (Advanced Use)

- Build unified API layer across Claude, OpenAI, Gemini
- Plug GenAI API into LangChain or FastAPI
- Auto-prompt chaining across APIs
- Use Weights & Biases for prompt monitoring

Would you like help with:

- □ Practice tasks for Claude / Gemini / OpenAI APIs
- □ A unique weekend GitHub project combining **multiple APIs**
- □ README + resume bullet + LinkedIn post template

Let me know — or ready to continue to **Week 6 – RAG Pipelines + Evaluation**?

Here's your full breakdown for **Week 6 – Advanced RAG Pipelines + Evaluation**. This week is all about making your RAG systems **more powerful, optimized, testable, and enterprise-ready** — beyond just "Chat with PDF".

□ WEEK 6 – RAG Pipelines + Evaluation

□ 1. Advanced RAG Architecture

Subtopics:

- Limitations of basic RAG (low retrieval quality, hallucination)
- RAG = Query → Embed → Retrieve → Prompt → Generate
- Two-stage RAG (retriever → reranker → LLM)
- Hybrid search (vector + keyword / BM25)
- Multi-query + metadata filters
- Language-aware / context-aware retrieval

□ 2. LangChain Evaluation Tools

Subtopics:

- Built-in LangChain `Evaluators` module
- Eval types: RetrievalRecallEvaluator, QA Eval, String Eval
- Compare model responses (e.g., GPT-4 vs Mistral vs Claude)
- Create your own eval dataset
- Using LLM-as-a-judge
- Logging + visualizing eval results

✂□ 3. Context Chunking Strategies

Subtopics:

- Why chunking is important in RAG
- RecursiveCharacterTextSplitter vs TokenTextSplitter
- Chunk size vs overlap – how to balance
- Adaptive chunking (semantic-aware)
- Splitting by sentence, heading, paragraph
- Chunk metadata tagging (title, section, source, date)
- Effects on retrieval accuracy

□ 4. Memory in LLM Applications

Subtopics:

- Difference: Memory vs Context vs Vector DB
- LangChain Memory Types:

    - ConversationBufferMemory
    - ConversationSummaryMemory
    - EntityMemory
- Memory with RAG (conversation-aware agents)
- When to use memory vs fresh query

□ 5. Cost Optimization in RAG Pipelines

Subtopics:

- Minimize tokens in prompt and context
- Reduce vector store size (deduplication, summarization)
- Rerank with smaller models before final LLM
- Use smaller embedding models (e.g., all-MiniLM)
- Cache frequent queries
- Use system prompts to guide shorter generation

□ 6. Retrieval Testing & Evaluation Metrics

Subtopics:

- Precision / Recall in retrieval
- Retrieval Recall@K
- MRR (Mean Reciprocal Rank)
- F1 for extractive QA
- BLEU, ROUGE, METEOR for text comparison
- Human judgment vs LLM judgment

□ 7. LLM Hallucination – Detection & Fixes

Subtopics:

- What causes hallucinations?
- How retrieval quality affects generation
- Adding citations + sources
- Post-generation filters
- Prompting with rules: “Don’t guess,” “Only use context”
- Guardrails: JSON schema, key-value enforcement

□ OPTIONAL ADVANCED (If time permits)

- Use `rerankers` (e.g., Cohere / Jina AI) before final answer
- Compare RAG with and without memory
- Use OpenAI Function Calling or LangChain Tools to ground responses

---

Would you like help with:

- □ Practice tasks (chunking tests, eval metrics demo, memory experiments)
- □ A **weekend GitHub project** that shows off full RAG evaluation pipeline
- □ A project README + resume bullet + LinkedIn post format

I can also move on to **Week 7 – Agentic AI (LangGraph, CrewAI, AutoGen)** next if you're ready!

Excellent — now we move to one of the most exciting and futuristic areas in your 12-week roadmap:

Week 7 – Agentic AI (LangGraph, CrewAI, AutoGen)

This week is all about turning **LLMs into agents** that can reason, plan, take actions, collaborate, and complete goals — with minimal human input. These are the **backbone of autonomous AI systems** being used by startups, research labs, and Big Tech.

□ WEEK 7 – Agentic AI (LangGraph / CrewAI / AutoGen)

□ 1. What Is Agentic AI?

Subtopics:

- What is an “AI agent” vs a normal chatbot?
- Autonomous vs semi-autonomous agents
- ReAct, Reflexion, AutoGPT, BabyAGI: historical evolution
- Use cases: research assistant, workflow automation, coding agents, AI CEO
- Agent loop: **Observe → Plan → Act → Learn**

□ 2. ReAct Prompting (Reason + Act)

Subtopics:

- What is ReAct (Reasoning + Acting)?
- ReAct prompt structure: Thoughts, Actions, Observations
- Tool use inside reasoning
- OpenAI tool-calling (functions API) + ReAct
- When to use ReAct over chain-of-thought or RAG
- ReAct in LangChain and CrewAI agents

□ 3. LangGraph Framework

Subtopics:

- What is LangGraph?
- Difference between LangChain and LangGraph
- State-machine-based agent flow
- Building cyclic and conditional agent flows
- Define edges: "if this then move to step B"
- Multi-step workflows with memory
- Use cases: conversational planner, agent decision engine

□ 4. CrewAI (Multi-Agent Collaboration)

Subtopics:

- What is CrewAI?
- Define agents with roles, goals, tools
- Coordinator vs worker agent model

- Task delegation between agents
- LangChain + CrewAI integration
- Prompt templating for roles (e.g., researcher, developer, tester)
- Use cases: multi-step reports, code generation & testing

□ 5. AutoGen Framework (by Microsoft)

Subtopics:

- What is AutoGen? (from Microsoft Research)
- ChatManager, AssistantAgent, UserProxyAgent
- Define agent roles + memory + tools
- Agent ↔ Agent conversation (debate, feedback loop)
- Code execution, external API calls
- Use AutoGen to simulate teams of agents
- When to use AutoGen vs CrewAI

□□ 6. Tools, Actions & Pluggable Functions

Subtopics:

- Agents with tools (calculator, search, PDF reader)
- LangChain `Tool` class
- OpenAI Function Calling vs LangChain tools
- Add your own Python tools to agents
- Toolchains: one agent → multiple tools → orchestrated actions

---

□ 7. Building Multi-Agent Workflows

Subtopics:

- Orchestrate tasks across agents
- Use roles: planner → researcher → executor → summarizer
- Handle failures, retries, timeouts
- Add vector DBs and memory between agent steps
- Monitoring agent conversation flows
- Saving logs and agent decisions

□ OPTIONAL ADVANCED

- Human-in-the-loop agents
- Scheduling agents via cron / queues
- Prompt-based role alignment and bias handling
- Chain of agents with frontend (e.g., Streamlit / Gradio)

Would you like me to prepare:

- □ Practice tasks for Thursday–Friday
- □ Weekend project: **"AI Startup Team" using CrewAI or AutoGen** (publish on GitHub)
- □ README, resume line, and demo video checklist

Let me know when ready — and I'll move to **Week 8 – Full AI App Integration & Deployment** after that!

Perfect — now you're entering **Week 8**, which is focused on **Full AI App Integration & Deployment**. This is where everything you've learned so far comes together: LLMs, vector DBs, APIs, agents, and UI — all wrapped in a deployable full-stack AI product.

□ WEEK 8 – Full AI App Integration & Deployment

□ 1. Flask + LangChain Backend for AI Apps

Subtopics:

- What is Flask and when to use it for AI projects
- Creating routes for chatbot, RAG, or agents
- Connecting LangChain chain to Flask endpoint

- Using `POST` and `GET` requests
- Returning responses in JSON format
- Handling file uploads (e.g. PDF for Chat with PDF)
- Deploy-ready folder structure (with `app.py`, `requirements.txt`)

□ 2. FastAPI for LLM & RAG APIs

Subtopics:

- Why FastAPI is better for scalable backend APIs
- Setting up endpoints for:
  - Chat
  - RAG
  - Agent workflows
- FastAPI + LangChain integration
- Swagger Docs + testing from browser
- Handling auth and headers
- Hosting LLM or vector DB API as microservice

□ 3. Vector DB + LLM Integration (End-to-End)

Subtopics:

- Full flow: Upload → Chunk → Embed → Store → Query → Respond
- Embedding using `SentenceTransformer` or OpenAI
- Store and search vectors using FAISS / Chroma / Pinecone
- LLM as the final generator using retrieved documents
- Implementing retrieval filters (metadata, time, source)
- Clean error handling and logging

□ 4. Frontend Integration (React / HTML / Streamlit)

Subtopics:

- Creating a basic frontend (optional: React or HTML + JS)
- Calling Flask/FastAPI from frontend via `fetch()` or `axios`
- Input box → submit → get AI response → render result
- Upload files, show summaries / answers
- Optional: use Streamlit for zero-code dashboards

□ 5. Deploy on Render / Railway / Vercel

Subtopics:

- What is Render, Railway, Vercel?
- Hosting Flask or FastAPI apps
- Creating `requirements.txt`, `Procfile`
- Free vs paid hosting (memory & request limits)
- Setup environment variables (API keys)
- Connect GitHub to CI/CD flow
- Monitor logs & restart on crash

☁□ 6. IBM Cloud Deployment (watsonx.ai Integration)

Subtopics:

- Hosting AI app using IBM Cloud App Services
- Upload model to IBM AI Studio
- Use IBM API keys inside Flask backend
- Connect IBM services (Cloud Object Storage, watsonx.ai API)
- Build a secure cloud-deployable enterprise app

□ 7. Fullstack GenAI Project Structure

Subtopics:

- Recommended folder structure for GenAI apps:
- `/backend`
- `  └── app.py`
- `  └── rag_pipeline.py`
- `  └── tools/`
- `/frontend`
- `  └── index.html / React files`
- `/data`
- `  └── sample.pdf`
- `requirements.txt`
- `README.md`
- Modularizing RAG, embeddings, memory, and agents
- Adding `.env` for secret keys
- Preparing your GitHub for public demo

□ OPTIONAL ADVANCED

- Dockerize the app for portable deployment
- Add user authentication (e.g., JWT)
- Add analytics or usage logging
- Convert into a Chrome extension or VS Code plugin frontend
- Setup auto-reload on update with GitHub Actions

Would you like me to:

- □ Give you **practice integration tasks** (RAG + API + frontend)?
- □ Help you build a **full GitHub project this weekend** (with deployable code)?
- □ Write your project README + deployment guide + resume bullet point?

Once done — you're ready for Week 9: **Train Your Own Models (LoRA / QLoRA / Dataset Building)**. Want me to begin Week 9 next?